

Peer-to-Peer Networks / Onion Routing

Anonymous Communication in Distributed Systems

Zhivko Stoimchev
89221056@student.upr.si
UP FAMNIT

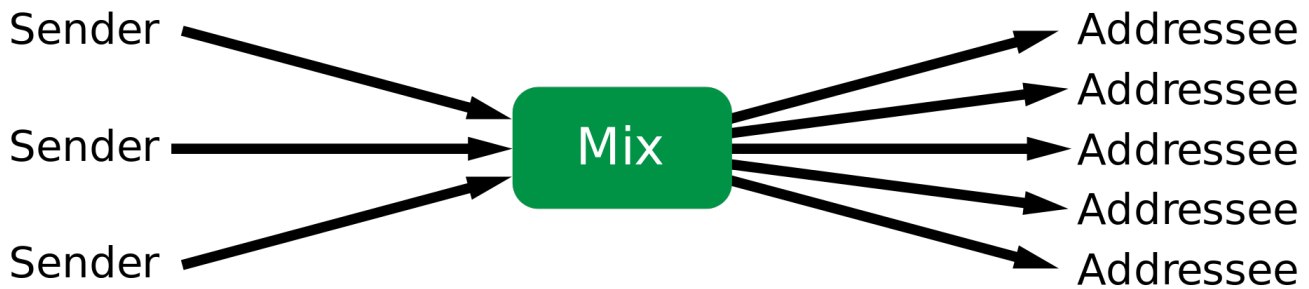


Figure 1: A mix node collecting messages and forwarding them, making it impossible to link them with the sender.

ABSTRACT

Growing concerns around online privacy and traffic analysis have made anonymous communication an increasingly relevant problem in modern networking. Standard internet traffic exposes the source and destination of every connection, allowing third parties to monitor user behaviour without ever reading the content itself. ZMIX is a peer-to-peer onion routing network implemented from scratch in Java 21 that addresses this problem directly.

The system builds a self-organizing overlay network through gossip-based peer discovery, then constructs multi-hop anonymous circuits using Elliptic Curve Diffie-Hellman key agreement. HTTP traffic is anonymized by wrapping it in nested layers of AES-256-GCM encryption — one per circuit hop — so that each relay node only decrypts its own layer and learns nothing beyond its immediate neighbours. The implementation covers a custom message protocol, concurrent peer and circuit management, a CLI for issuing requests, and a Docker Compose setup for simulating a distributed network environment.

KEYWORDS

Onion Routing, Peer-to-Peer Networks, Anonymous Communication, Network Security, Public Key Cryptography, Docker, Java 21

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Programming III - Concurrent programming, May 2026, FAMNIT
© 2026 Association for Computing Machinery.

ACM Reference Format:

Zhivko Stoimchev. 2026. Peer-to-Peer Networks / Onion Routing: Anonymous Communication in Distributed Systems. In *Proceedings of Peer-to-Peer Networking Course Project (Programming III - Concurrent programming)*. ACM, New York, NY, USA, 5 pages.

1 INTRODUCTION

Traditional internet communication, even when encrypted, exposes metadata that reveals who is talking to whom, how often, and for how long. This information alone is enough for traffic analysis and identity disclosure, without ever breaking the encryption on the content. Onion routing solves this by routing traffic through a chain of intermediary nodes called a circuit, where each node only knows its immediate predecessor and successor. The message is wrapped in multiple layers of encryption — one per hop — and each node peels exactly one layer before forwarding the rest. No single node ever sees both the origin and the destination at the same time.

ZMIX implements this concept over a fully decentralized peer-to-peer network. Nodes discover each other through a gossip protocol, negotiate per-hop session keys using ECDH, and route HTTP requests through layered AES-256-GCM encryption. The project is built entirely in Java 21 with no external cryptography libraries, and uses Docker Compose to simulate a live multi-node environment. The rest of the report covers the system architecture in Section 2, the protocol design and circuit construction in Section 3, the security design in Section 4, data transfer in Section 5, deployment in Section 6, and limitations and future work in Section 7.

2 SYSTEM ARCHITECTURE

ZMIX is a fully decentralized overlay network where every node runs the same binary. There are no fixed roles — whether a node acts as a relay, exit node, or circuit initiator depends entirely on the network state at runtime. The only exception is the bootstrap

node, which serves as the initial seed for peer discovery and does not attempt outbound connections on startup.

Internally the design is split into three layers: transport (TCP sockets and message serialization), network management (peer registry and connection maintenance), and protocol logic (peer discovery and circuit operations). These layers communicate through a shared event bus — a `BlockingQueue<Event>` — so that I/O threads never directly invoke protocol logic. Every received message is placed into the queue and processed by a single dedicated consumer thread.

2.1 Component Overview

The main components of the system are:

- **Server** – Listens for inbound TCP connections and spawns a `Peer` thread for each accepted connection.
- **Peer** – Handles the handshake, reads inbound messages, and places them onto the shared event queue.
- **NetworkManager** – Central registry for connected and known peers. Drives scheduled connection maintenance.
- **MessageHandler** – Single-threaded event queue consumer. Dispatches each message to the appropriate protocol handler.
- **PeerDiscoveryProtocol** – Handles peer list exchange and periodically broadcasts discovery requests.
- **CircuitManager** – Manages circuit state and handles all circuit-related messages.
- **Crypto** – Provides ECDH key agreement, SHA-256 key derivation, and AES-256-GCM encryption and decryption.
- **Config** – Loads `.properties` configuration files and applies environment variable overrides for Docker deployments.
- **Cli** – User interface for submitting URLs into the circuit request queue via the terminal.

2.2 Thread Model

Each peer connection runs on its own thread from a thread pool `CachedThreadPool`. All protocol logic runs on the single message handler, namely `MessageHandler` thread, which means handlers in `CircuitManager` and `PeerDiscoveryProtocol` do not need internal synchronization — they are never called concurrently. Circuit construction runs on `SingleThreadScheduledExecutor`, a separate thread to avoid blocking peer I/O, and once the circuit is active a dedicated `requestExecutor` processes queued URLs one at a time. Two scheduled tasks run in the background: one for periodic peer discovery broadcasts and one for connection maintenance.

3 PROTOCOL DESIGN

3.1 Message Format

All communication between nodes uses a custom text-based protocol over persistent TCP connections. Each message is a single newline-terminated string with four fields separated by a fixed delimiter:

```
MessageType ;delim;;; Timestamp ;delim;;; MessageId ;delim;;; Payload
```

The delimiter `;delim;;;` was chosen to avoid collisions with Base64 or UUID characters that appear in the payload fields. Binary data such as public keys and encrypted bytes is Base64-encoded

before embedding. Each payload type implements a `toBytes()` and `fromBytes()` contract, keeping serialization logic inside the payload class itself.

3.2 Message Types

- **HANDSHAKE** – First message on every new connection. Carries the sender's public key and listening port.
- **PEER_DISCOVERY_REQUEST** – Requests the recipient's known peer list. No payload.
- **PEER_DISCOVERY_RESPONSE** – Returns a list of known peers in the format `publicKey;host;port`.
- **CIRCUIT_CREATE_REQUEST** – Initiates a key exchange. Carries a circuit ID and an ephemeral public key.
- **CIRCUIT_CREATE_RESPONSE** – Returns the target node's ephemeral public key to complete the exchange.
- **CIRCUIT_EXTEND_REQUEST** – Tunnels a circuit creation request to the next hop through the existing circuit.
- **CIRCUIT_EXTEND_RESPONSE** – Returns the next hop's ephemeral key back to the initiator.
- **DATA_TRANSFER_REQUEST** – Sends a layered-encrypted HTTP request through the circuit.
- **DATA_TRANSFER_RESPONSE** – Returns the HTTP response back along the reverse path.

3.3 Handshake & Peer Registration

Every new TCP connection starts with a handshake before any other message is processed. The outbound node sends its **HANDSHAKE** first; the inbound node waits, validates it, and then sends its own. A five-second socket timeout guards against hung connections and is reset to zero once the handshake completes.

The handshake payload carries the node's long-term EC public key and its listening port. This is necessary because the socket's remote port is the ephemeral OS-assigned port, not the actual listening port — so it must be communicated explicitly.

After a successful handshake the peer is registered in `NetworkManager`. If the node has already reached its maximum connection limit, it sends the new peer a short list of known peers and disconnects immediately, acting as a lightweight referral.

3.4 Peer Discovery

Peer discovery uses a simple gossip protocol seeded by the bootstrap node. When an outbound connection completes its handshake, the peer immediately sends a **PEER_DISCOVERY_REQUEST**. The receiving node replies with its full known peer list, and the requester merges any new entries, filtering out itself and peers it already knows.

Two scheduled tasks keep the topology healthy over time. `PeerDiscoveryProtocol` periodically broadcasts discovery requests to all connected peers, and `NetworkManager.startPeerMaintenance()` attempts outbound connections to known-but-disconnected peers until the minimum connection floor (`node.connections.min`) is met.

3.5 Circuit Construction

Circuit construction is the most important part of the protocol. Rather than sending all routing information in a single message, the initiator builds the circuit incrementally, negotiating a shared secret

```

1 byte[] encrypted = payload.toBytes();
2 for (int i = hop - 1; i >= 0; i--) {
3     encrypted = crypto.encryptAES(encrypted, keys.
4         get(i));
5 }

```

Listing 1: Encrypting the extend payload through established hops

with each hop one at a time through the hops already established. This ensures no relay ever learns the full path.

For a three-hop circuit through nodes E (entry), M (middle), and X (exit):

Step 1 — Entry node. The initiator connects to E and sends a `CIRCUIT_CREATE_REQUEST` with a fresh ephemeral EC public key. E replies with its own ephemeral key. Both sides perform ECDH and independently derive the same session key K_0 .

Step 2 — Middle node. The initiator wraps M's address and a new ephemeral key inside a payload, encrypts it with K_0 , and sends it as a `CIRCUIT_EXTEND_REQUEST` to E. E decrypts, connects to M, and forwards a create request on the initiator's behalf. M's key travels back through E re-encrypted with K_0 , and the initiator derives K_1 .

Step 3 — Exit node. The same process repeats for X, but the payload is encrypted first with K_1 then with K_0 . E peels K_0 , M peels K_1 , and X receives the plaintext request. X's response travels back re-encrypted at each relay, and the initiator derives K_2 .

Once all three exchanges complete, the circuit status is set to `ACTIVE` and the initiator holds $[K_0, K_1, K_2]$, while each relay only knows its own key and its immediate neighbours.

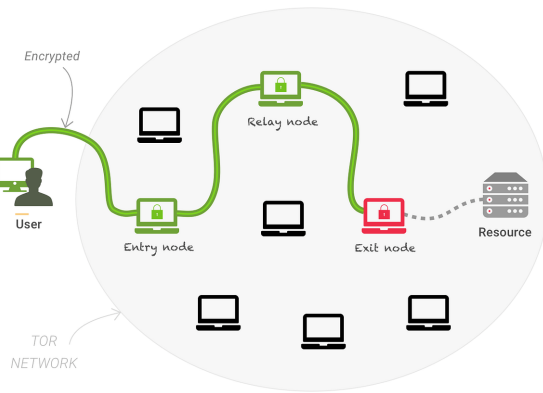


Figure 2: Three-hop circuit construction. The initiator (A) negotiates a separate session key with each relay without any single node knowing the full path.

4 SECURITY DESIGN

4.1 Key Agreement and Derivation

Each hop in the circuit gets its own independent session key negotiated using Elliptic Curve Diffie-Hellman on the `secp256r1` (P-256) curve. Both sides generate a fresh ephemeral key pair per hop,

```

1 byte[] encrypted = requestBytes;
2 for (int i = currentHop - 1; i >= 0; i--) {
3     encrypted = crypto.encryptAES(encrypted, keys.
4         get(i));
5 }

```

Listing 2: Layered encryption of the outgoing HTTP request

exchange public keys through the circuit messages, and perform ECDH locally to arrive at the same shared secret without transmitting it. The raw ECDH output is then passed through SHA-256 and the first 32 bytes are used as the AES-256 key.

The node's long-term identity key is only used during the handshake. It plays no role in encrypting circuit data.

4.2 Symmetric Encryption

All data encryption uses AES-256-GCM. GCM is an authenticated encryption mode, meaning it provides both confidentiality and integrity in one operation. Every encryption call generates a fresh random 12-byte IV using `SecureRandom`, which is prepended to the ciphertext before sending. If a relay tampers with a payload in transit, decryption throws an exception rather than silently returning corrupted data.

4.3 Forward Secrecy

Because each circuit uses freshly generated ephemeral key pairs that are discarded after the session key is derived, compromising a node's long-term private key later does not expose any past session keys. An attacker who records all traffic and later breaks into a relay gains nothing useful from the recorded data.

5 DATA TRANSFER

5.1 Sending a Request

Once the circuit is `ACTIVE`, the user submits a URL through the CLI. A raw HTTP/1.1 GET request is built and encrypted in layers starting from the exit node's key and working backwards to the entry node's key. The entry node's layer is on the outside; only the exit node will see the plaintext request after all layers are peeled:

The wrapped payload is sent as a `DATA_TRANSFER_REQUEST` to the entry node, carrying the circuit ID, target host and port, and the encrypted data.

5.2 Relay Forwarding and the Exit Node

Each relay decrypts one layer and checks whether it has a next hop. If it does, it forwards the message onward. If there is no next hop, the node is the exit node and holds the original plaintext HTTP request.

The exit node opens a direct TCP connection to the target. Port 443 uses Java's `SSLSocketFactory` for HTTPS; everything else uses a plain socket. It writes the request, reads the full response, encrypts it with its session key, and sends it back as a `DATA_TRANSFER_RESPONSE`.

```

1 java -jar target/zmix-1.0-SNAPSHOT.jar node.
  bootstrap.properties
2 java -jar target/zmix-1.0-SNAPSHOT.jar node.peer.
  test1.properties
3 java -jar target/zmix-1.0-SNAPSHOT.jar node.peer.
  test2.properties
4 java -jar target/zmix-1.0-SNAPSHOT.jar node.peer.
  test3.properties

```

Listing 3: Starting nodes locally

```

1 docker-compose up --build
2 docker attach peer1

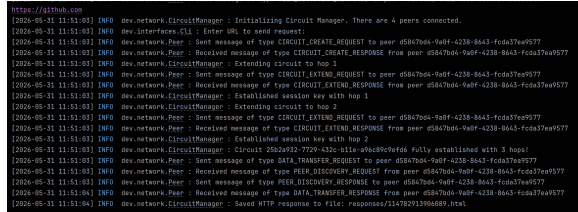
```

Listing 4: Starting the full network with Docker

5.3 Receiving the Response

The response travels back in reverse. Each relay re-encrypts with its own key before passing it upstream. The initiator peels the layers in order — entry first, then middle, then exit — recovering the original HTTP response. The HTML body is extracted and saved to the responses/ directory as a timestamped .html file.

When tested with three peers running locally, the circuit establishes in under a second and a request to a public website returns a valid HTML response within a few seconds depending on the target server. The log output confirms each hop decrypts and forwards correctly.



```

2026-05-31 11:51:03 INFO dev.network.CircuitManager: Initializing Circuit Manager. There are 4 peers connected.
2026-05-31 11:51:03 INFO dev.interfaces.Cli: Enter URL to send request:
2026-05-31 11:51:03 INFO dev.network.Peer: Sent message of type CIRCUIT_CREATE_REQUEST to peer d5847b04-9a0f-4238-8643-fcda37ae9577
2026-05-31 11:51:03 INFO dev.network.Peer: Received message of type CIRCUIT_CREATE_RESPONSE from peer d5847b04-9a0f-4238-8643-fcda37ae9577
2026-05-31 11:51:03 INFO dev.network.CircuitManager: Extending circuit to hop 1
2026-05-31 11:51:03 INFO dev.network.Peer: Sent message of type CIRCUIT_EXTEND_REQUEST to peer d5847b04-9a0f-4238-8643-fcda37ae9577
2026-05-31 11:51:03 INFO dev.network.Peer: Received message of type CIRCUIT_EXTEND_RESPONSE from peer d5847b04-9a0f-4238-8643-fcda37ae9577
2026-05-31 11:51:03 INFO dev.network.CircuitManager: Established session key with hop 1
2026-05-31 11:51:03 INFO dev.network.CircuitManager: Extending circuit to hop 2
2026-05-31 11:51:03 INFO dev.network.Peer: Sent message of type CIRCUIT_EXTEND_REQUEST to peer d5847b04-9a0f-4238-8643-fcda37ae9577
2026-05-31 11:51:03 INFO dev.network.Peer: Received message of type CIRCUIT_EXTEND_RESPONSE from peer d5847b04-9a0f-4238-8643-fcda37ae9577
2026-05-31 11:51:03 INFO dev.network.CircuitManager: Established session key with hop 2
2026-05-31 11:51:03 INFO dev.network.CircuitManager: Circuit 252a9272-7729-432c-b1fe-9bdc8c9c6fda fully established with 3 hops!
2026-05-31 11:51:03 INFO dev.network.Peer: Sent message of type DATA_TRANSFER_REQUEST to peer d5847b04-9a0f-4238-8643-fcda37ae9577
2026-05-31 11:51:03 INFO dev.network.Peer: Received message of type DATA_TRANSFER_RESPONSE from peer d5847b04-9a0f-4238-8643-fcda37ae9577
2026-05-31 11:51:03 INFO dev.network.Peer: Sent message of type PEER_DISCONNECT_REQUEST to peer d5847b04-9a0f-4238-8643-fcda37ae9577
2026-05-31 11:51:03 INFO dev.network.Peer: Received message of type PEER_DISCONNECT_RESPONSE from peer d5847b04-9a0f-4238-8643-fcda37ae9577
2026-05-31 11:51:04 INFO dev.network.CircuitManager: Saved HTTP response to file: responses/214782113704a007.html

```

Figure 3: Terminal output confirming a three-hop circuit was established and an HTTP response was successfully returned.

6 SETUP & DEPLOYMENT

6.1 Running Locally

The project builds with mvn package which produces a fat JAR via the maven-shade-plugin. Nodes are started by passing a properties file as the first argument. The bootstrap node must be started first:

Once at least three peers are connected and the circuit is established, a URL can be entered directly in the terminal.

6.2 Docker Compose

The included docker-compose.yml defines four services: bootstrap-node with port 12137 exposed to the host, and peer1, peer2, peer3 all on a shared bridge network. The Dockerfile performs a two-stage build: Maven compiles the fat JAR in the builder stage and Eclipse Temurin 21 runs it in the final stage.

Environment variables NODE_PORT, NODE_BOOTSTRAP, and BOOTSTRAP_HOST override the properties file values so the same image works as either a bootstrap node or a regular peer depending on the Compose service definition.

6.3 Configuration

The key properties and their defaults are:

- node.port – TCP listening port (default: 12137).
- node.bootstrap – If true, acts as the seed node (default: false).
- node.connections.max – Maximum simultaneous peers (default: 5).
- node.connections.min – Maintenance connection floor (default: 3).
- circuit.length – Number of hops per circuit (default: 3).
- bootstrap.host / bootstrap.port – Address of the seed node to connect to on startup.
- connection.retry.attempts – Max reconnect attempts (default: 10).

7 LIMITATIONS & FUTURE WORK

The implementation covers all the core mechanics of onion routing and works correctly in a local multi-node setup. There are however several known limitations worth addressing before any real-world use.

7.1 Current Limitations

- **Single circuit per node.** Only one outbound circuit is supported at a time. Concurrent requests queue behind it.
- **No circuit teardown.** There is no CIRCUIT_DESTROY message. Relay entries accumulate in memory if the initiator disconnects unexpectedly.
- **Unbounded replay cache.** The message ID history in MessageHandler is an ArrayList that grows without limit. A bounded LRU cache or Bloom filter would be more appropriate for long-running nodes.
- **Simplified key derivation.** SHA-256 applied directly to the ECDH output works but HKDF (RFC 5869) with proper salt and context is the correct approach.
- **No circuit rotation.** The same circuit is reused for all requests until the node restarts, which weakens long-term anonymity.

7.2 Future Work

- Support multiple concurrent circuits with a circuit pool.
- Implement CIRCUIT_DESTROY and relay garbage collection.
- Replace the SHA-256 KDF with proper HKDF.
- Add periodic circuit rotation to limit traffic correlation.
- Add unit tests for the serialization layer and crypto round-trips.

8 CONCLUSION

ZMIX demonstrates that the core mechanics of onion routing can be built from scratch in plain Java without any external frameworks. The system successfully anonymizes HTTP traffic across a self-organizing peer-to-peer network — no single node can determine

both the origin and destination of a request, which is the fundamental privacy guarantee onion routing is designed to provide.

The design is modular and easy to reason about: the event-driven message bus keeps I/O and protocol logic separate, and the protocol registration pattern makes adding new message types straightforward. The Docker Compose deployment shows everything works in a real multi-container environment with no code changes.

The limitations are well understood and addressable. As a proof of concept the project covers all the essential building blocks: peer discovery, authenticated key exchange, layered encryption, and end-to-end anonymous HTTP proxying.

REFERENCES